

Valores XP del proyecto

Marco conceptual

Extreme Programming define cinco valores que constituyen la base filosófica de todas sus prácticas. Los valores no son slogans: son el criterio que permite decidir cuándo una práctica específica aplica y cuándo no. Sin valores compartidos, las prácticas XP se aplican mecánicamente y pierden su efecto.

En el proyecto “Sistema de Recomendación de Carrera Universitaria para UVG Campus Central 2026”, los valores XP operan durante la Fase 2 (1 de enero 2027 – 22 de abril 2027), cuando el equipo construye el sistema en 8 sprints de dos semanas. El equipo tiene 5 personas: Josué Say (PO, 0.7 FTE), Mathew Cordero (SM), y tres developers senior —Gustavo Cruz Bardales, Javier Chen Gonzalez, Pedro Guzmán Mayen (1.0 FTE cada uno).

Los cinco valores

1. Comunicación

Definición. El conocimiento técnico no es propiedad individual. Cuando un developer entiende algo que los demás no entienden, el equipo tiene un punto de falla. La comunicación en XP no es solo hablar: es transferir conocimiento técnico de forma continua y estructurada.

Aplicación en el proyecto. Los dos mecanismos principales son:

- **Pair Programming:** dos developers trabajan en el mismo problema al mismo tiempo. Uno escribe (driver), el otro revisa en tiempo real (navigator). Los roles rotan. Este esquema no es una reunión: es transferencia de conocimiento integrada al acto de escribir código.
- **Collective Ownership:** cualquier developer puede leer, entender y modificar cualquier parte del código base. La comunicación técnica se hace permanente a través del código compartido.

Coherencia con el contexto. El equipo tiene tres developers senior. La comunicación horizontal entre pares —no la supervisión vertical— es la forma natural en la que tres personas de nivel equivalente resuelven problemas complejos. Pair Programming y Collective Ownership respetan esa horizontalidad.

Práctica que lo materializa: Pair Programming, Collective Ownership.

2. Simplicidad

Definición. La solución más simple que cumple el criterio de aceptación es la correcta. No la más elegante, no la más general, no la que resuelve problemas futuros hipotéticos. La que resuelve exactamente lo que la historia de usuario requiere hoy.

Aplicación en el proyecto. El valor de Simplicidad es crítico en EP04 (motor de recomendación). La tentación en un motor de recomendación es diseñarlo para ser generalizable: que pueda usarse para otras universidades, otros años académicos, otros contextos. Ese diseño no está en el alcance. El motor de recomendación de este proyecto es para el pensum 2026 de UVG Campus Central, y debe diseñarse solo para ese caso.

La misma lógica aplica a EP05 (ponderación por afinidad): el modelo de ponderación más simple que produce resultados deterministas para perfiles iguales es el correcto.

Coherencia con el contexto. El proyecto tiene 8 sprints y fecha de entrega fija (22 de abril 2027). La complejidad innecesaria consume capacidad del equipo que podría usarse en funcionalidad real. Simple Design no es descuido: es una decisión estratégica de alcance.

Práctica que lo materializa: Simple Design, TDD (las pruebas definen exactamente qué debe hacer el código, nada más).

3. Retroalimentación

Definición. El equipo necesita señales frecuentes de si está construyendo lo correcto y si lo está construyendo bien. La retroalimentación tardía es costosa: cuanto más tarde se descubre un problema, más código debe cambiarse.

Aplicación en el proyecto. La retroalimentación opera en tres ciclos de distinta frecuencia:

- **TDD (ciclos de minutos):** la prueba que falla es retroalimentación inmediata sobre si el código hace lo que se prometió. El ciclo Red-Green-Refactor provee señal en el orden de minutos.
- **CI (ciclos de horas):** el pipeline de integración continua se ejecuta en cada push al repositorio principal. Los problemas de integración se detectan en horas, no en semanas. Esto es especialmente relevante para R2 (riesgo de retraso técnico cloud, probabilidad 0.40): la integración con servicios cloud se verifica automáticamente en cada ciclo.
- **Sprint Review (ciclos de dos semanas):** el PO y los stakeholders de UVG Campus Central ven el incremento funcionando y proveen retroalimentación sobre si el sistema cumple las necesidades reales de orientación vocacional.

Coherencia con el contexto. Un equipo de cinco personas que trabaja en un sistema de información académica con fecha de entrega fija no puede permitirse descubrir problemas de integración en la última semana del proyecto. Los tres ciclos de retroalimentación garantizan que los problemas se detectan cuando aún hay capacidad para resolverlos.

Práctica que lo materializa: TDD, Continuous Integration, Small Releases.

4. Coraje

Definición. Refactorizar código que funciona requiere coraje. Decir “esto necesita reescribirse” cuando hay presión por avanzar requiere coraje. No esconder deuda técnica bajo código que “parece funcionar” requiere coraje.

Aplicación en el proyecto. El coraje se manifiesta en dos contextos específicos:

- **Refactoring continuo:** el equipo está comprometido a refactorizar código funcional cuando la estructura interna lo necesita. Esto es parte del DoD (criterio XP-3: sin deuda técnica deliberada), no una tarea opcional. Los sprints 5 al 7 concentran complejidad técnica alta (EP04, EP05, EP06) y son el momento donde la presión de avanzar es mayor. El coraje es lo que impide que esa presión acumule deuda.
- **Deuda técnica visible:** cuando el equipo identifica deuda técnica durante el desarrollo, la registra en el backlog como ítem explícito. No la esconde, no la etiqueta como “lo arreglo después”. Si la solución correcta no cabe en el sprint, la historia vuelve al backlog.

La frase del docente aplica directamente aquí: “La deuda técnica no se ve hasta que paraliza el proyecto.”

Coherencia con el contexto. Tres developers senior tienen el criterio técnico para identificar cuándo una solución es inadecuada. El coraje es el valor que convierte ese criterio en acción, incluso bajo presión de tiempo o de velocidad de sprint.

Práctica que lo materializa: Refactoring, DoD criterio XP-3.

5. Respeto

Definición. El respeto en XP no es un valor genérico de trabajo en equipo. Tiene dos manifestaciones técnicas concretas: Pair Programming como colaboración horizontal (no supervisión) y Collective Ownership como respeto por el código escrito por otros.

Aplicación en el proyecto. Pair Programming en este proyecto se entiende como dos developers senior de nivel equivalente que resuelven un problema juntos. No es un senior que supervisa el trabajo de otro: es colaboración horizontal. Esto es especialmente relevante en EP04-US01 (algoritmo de ranking por afinidad, 8 puntos de historia, sprint 5) y EP05-US01 (ponderación por afinidad, 8 puntos de historia, sprint 6): código de alta criticidad donde la revisión en tiempo real reduce defectos y distribuye conocimiento.

Collective Ownership implica que cuando un developer modifica código escrito por otro, lo hace con el mismo cuidado con que modificaría el propio. Los estándares de codificación acordados (HT-02, sprint 1) son el marco técnico que hace posible ese respeto sin ambigüedad sobre convenciones.

Coherencia con el contexto. Un equipo donde el código “le pertenece” a un individuo tiene un punto de falla cuando ese individuo no está disponible (R5, rotación de personal, probabilidad 0.20). El respeto como valor —materializado en Collective Ownership— es también la mitigación técnica de ese riesgo.

Práctica que lo materializa: Pair Programming, Collective Ownership.

Resumen: valores y prácticas

Valor	Práctica principal que lo materializa
Comunicación	Pair Programming, Collective Ownership
Simplicidad	Simple Design, TDD
Retroalimentación	TDD, Continuous Integration, Small Releases
Coraje	Refactoring, DoD XP-3
Respeto	Pair Programming, Collective Ownership

Los cinco valores no son independientes. El coraje para refactorizar es posible porque el respeto garantiza que los otros developers entienden y apoyan esa decisión. La simplicidad del diseño está respaldada por la retroalimentación de las pruebas. La comunicación que genera Pair Programming construye el respeto necesario para Collective Ownership.